

DOCUMENTACIÓN TÉCNICA

MISI FRAMEWORK
Versión 1.3.0
Estado: Final

Autor(a): Emily Monterrosa

Barranquilla, Colombia – 01 septiembre de 2026

ÍNDICE DE CONTENIDO

1. INTRODUCCIÓN 5
1.1 Propósito del documento 5
1.2 Alcance 5
1.3 Público objetivo 5
1.4 Convenciones utilizadas 5
2. VISIÓN GENERAL DEL PROYECTO 5
2.1 Descripción general 5
2.2 Propósito 5
2.3 Alcance actual 5
2.4 Limitaciones conocidas 6
3. STACK TECNOLÓGICO 6
3.1 Backend 6
3.2 Frontend 7
3.3 Base de datos 7
3.4 Infraestructura y despliegue 7
3.5 Herramientas de desarrollo 7
3.6 Testing, CI/CD y servicios externos 8
4. ARQUITECTURA DEL SISTEMA 8
4.1 Arquitectura general 8
4.2 Componentes principales 8
4.3 Comunicación entre componentes 9
4.4 Flujo general de datos 9
4.5 Arquitectura en capas del proyecto 10
4.6 Decisiones arquitectónicas identificables 10
5. ESTRUCTURA DEL PROYECTO 10
6. ARQUITECTURA DEL FRONTEND 12
Organización de páginas 12
Routing de vistas 12
Componentes, formularios y validación 12
Cliente API y assets 12
7. ARQUITECTURA DEL BACKEND 13
Controladores 13
Middleware 13
Manejo de errores 13
Procesos en segundo plano 13
8. MÓDULOS DEL SISTEMA 13
8.1 Módulo Catalog 13

8.2 Módulo Example 14
9. BASE DE DATOS 14
9.1 Tecnología 14
9.2 Modelo general 14
9.3 Entidades principales y relaciones 15
9.4 Tablas o colecciones 15
9.5 Campos importantes, claves y restricciones 16
9.6 Migraciones 16
9.7 Seeders 16
9.8 Índices y optimizaciones 16
10. API 17
10.1 Descripción general 17
10.2 Convenciones 17
10.3 Autenticación 17
10.4 Endpoints (rutas raíz — routes/web.php) 17
10.5 Manejo de errores 18
11. AUTENTICACIÓN Y AUTORIZACIÓN 18
11.1 Método 18
11.2 Flujo 18
11.3 Autorización (RBAC) 18
11.4 Protección de rutas y middleware 18
11.5 Cookies de sesión 18
12. INTEGRACIONES EXTERNAS 19
13. TESTING 19
14. BUILD Y DESPLIEGUE 19
14.1 Proceso de build 19
14.2 Despliegue objetivo: hosting compartido 19
14.3 Variable DEPLOY_TOKEN 19
14.4 Migraciones de producción 19
15. CONFIGURACIÓN DEL PROYECTO 20
15.1 Requisitos previos 20
15.2 Dependencias 20
15.3 Variables de entorno 20
15.4 Configuración local 21
16. Instalación y ejecución 21
16.1 Instalación global de la CLI (opcional) 21
16.2 Crear un proyecto nuevo 21

16.3 Configurar variables de entorno 21
16.4 Ejecutar migraciones 21
16.5 Cargar datos iniciales (opcional) 21
16.6 Ejecutar el proyecto en desarrollo 21
16.7 Diagnóstico del entorno 21
17. CLI DE DESARROLLO 22
18. INFRAESTRUCTURA Y OPERACIÓN 22
18.1 Almacenamiento de archivos 22
18.2 Logging 22
18.3 Monitoreo 23
18.4 Networking / dominios 23
19. SEGURIDAD 23
19.1 Medidas implementadas 23
19.2 Ausencias explícitas (documentadas por el propio código) 23
19.3 Hallazgos técnicos y recomendaciones 23
20. DEUDA TÉCNICA Y HALLAZGOS 24
21. LIMITACIONES Y ÁREAS PENDIENTES 25
22. GLOSARIO 25
23. REFERENCIAS 26
RESUMEN FINAL DE LA AUDITORÍA 27

1. INTRODUCCIÓN

1.1 Propósito del documento

Este documento describe la arquitectura, el stack tecnológico, la estructura de código, el modelo de datos, la seguridad, la instalación y el estado real de implementación de Misi Framework, a partir del análisis directo del código fuente contenido en el archivo comprimido proporcionado. No documenta funcionalidades planeadas que no tengan evidencia de implementación en el repositorio.

1.2 Alcance

El documento cubre el framework en sí (namespace `Misi\`), la capa de Business Core reutilizable (`Misi\Business\`), la aplicación de referencia incluida (`App\`) y los módulos de ejemplo (`Modules\Catalog` y `Modules\Example`). No incluye manual de usuario final ni material de capacitación comercial, conforme a la exclusión solicitada.

1.3 Público objetivo

- Desarrolladores que continúen o extiendan Misi Framework.
- Arquitectos de software que evalúen su adopción para un nuevo proyecto.
- Administradores técnicos responsables del despliegue en hosting compartido.
- Cualquier persona que mantenga aplicaciones construidas sobre este framework.

1.4 Convenciones utilizadas

Los espacios de nombres siguen PSR-4: `Misi\` para el núcleo del framework (carpeta `framework/`), `Misi\Business\` para la capa de negocio reutilizable (carpeta `business/`), `App\` para la aplicación de referencia (carpeta `app/`) y Modules\ para los módulos opcionales (carpeta `modules/`). Las migraciones se numeran con un prefijo secuencial (001_, 002_...) que determina su orden de ejecución dentro de cada fuente. Los permisos siguen la convención “recurso.accion” (por ejemplo, `products.manage`).

2. VISIÓN GENERAL DEL PROYECTO

2.1 Descripción general

Misi es un framework PHP propio, descrito por su propio README como “nuestro propio framework de desarrollo PHP para construir rápidamente sistemas administrativos para pequeños negocios; sin sobre-ingeniería, sin relleno”. No pretende competir con Laravel o Symfony: es una base técnica interna pensada para que cada proyecto nuevo arranque con la infraestructura común ya resuelta (configuración, ruteo, HTTP, base de datos, autenticación, autorización, validación, sesiones, seguridad, almacenamiento, logging y una CLI de desarrollo) y el desarrollo se concentre en la lógica específica de cada negocio.

2.2 Propósito

Evitar reconstruir la infraestructura técnica común cada vez que se inicia un proyecto administrativo o comercial pequeño, y estar optimizado para desplegarse en hosting compartido tradicional (FTP + MySQL/MariaDB + PHP 8+), sin Docker, sin SSH y sin procesos persistentes.

2.3 Alcance actual

Con base en el código analizado, el proyecto implementa de forma verificable:

- Un router HTTP propio con parámetros de ruta y middleware por alias (auth, guest, csrf).
- Una capa de acceso a base de datos (wrapper delgado sobre PDO/MySQL), sin ORM ni Query Builder.
- Un sistema de migraciones y seeders con soporte para migraciones de módulos.
- Autenticación basada en sesión PHP nativa y autorización mediante roles y permisos (RBAC simple).
- Un validador de entrada con reglas encadenables, incluyendo reglas que consultan la base de datos (unique, exists).
- Protección CSRF basada en token de sesión.
- Almacenamiento local de archivos con protecciones de path traversal y nombres aleatorios.
- Un logger a archivo con niveles y redacción automática de claves sensibles.
- Un sistema de módulos opcionales (`Modules\`) con rutas y migraciones propias, descubiertos automáticamente.
- Una capa "Business Core" reutilizable (clientes, categorías, productos) compartida entre proyectos.
- Un módulo de referencia (Catalog) que demuestra un panel administrativo protegido sobre Business Core.
- Una CLI de desarrollo (`bin/biz`, expuesta también como `misi/biz`) con comandos para crear proyectos, migrar, sembrar datos, servir localmente y generar código.

2.4 Limitaciones conocidas

El propio README documenta explícitamente, en su sección "Roadmap and Frozen Features", un conjunto de capacidades pospuestas de forma deliberada:

- No hay Query Builder ni ORM (se usa SQL crudo directamente).
- No hay abstracciones de ruteo adicionales (rutas nombradas con generación avanzada de URL, cacheo de rutas, subdominios).
- No hay contenedor de inyección de dependencias (los controladores se instancian con new $class() directamente).
- No hay sistema de eventos/hooks entre módulos.
- Solo existe un driver de almacenamiento (local); S3Storage/CloudStorage están definidos como interfaz, pero no implementados.
- No se implementan JWT, OAuth ni "remember me" persistente en Auth — solo sesión PHP nativa.

3. STACK TECNOLÓGICO

El stack se determinó a partir de composer.json, los badges e indicaciones del README, y el código fuente. No se encontró package.json ni ninguna dependencia de Node.js para producción.

3.1 Backend

Tecnología	Categoría	Versión	Propósito
PHP	Lenguaje	>= 8.1 (composer.json); README indica "PHP 8+"	Lenguaje de implementación del framework y la aplicación
Extensión mbstring	Extensión PHP	Requerida (composer.json,	Longitud de cadenas segura con UTF-8 (usada por Validator)

		verificada en bootstrap/autoload.php)	
Extensión PDO / pdo_mysql	Extensión PHP	Implícita (no declarada en composer.json, pero requerida en tiempo de ejecución)	Acceso a MySQL/MariaDB desde framework/Database/Database.php
Arquitectura propia "Misi\"	Micro-framework interno	1.4.0	Config, Routing, HTTP, Database, Auth, Validation, Security, Storage, Logging, Exceptions

3.2 Frontend

Tecnología	Categoría	Propósito
HTML5	Markup	Vistas PHP planas (resources/views/*.php) y páginas estáticas de marketing (public/views/*.html)
CSS3	Estilos	Hoja de estilos propia public/css/misi.css (344 líneas)
JavaScript (vanilla)	Cliente	Scripts propios sin framework (public/js/api.js, ui.js, uikit.js, catalog.js, welcome.js, landing.js)

3.3 Base de datos

Tecnología	Categoría	Versión	Propósito
MySQL / MariaDB	Motor de base de datos	No especificada; charset utf8mb4	Persistencia de datos (config/database.php, DSN mysql: en Database.php)
PDO (prepared statements)	Acceso a datos	Nativo de PHP	Todas las consultas usan sentencias preparadas; no hay ORM

3.4 Infraestructura y despliegue

Tecnología	Categoría	Propósito
Hosting compartido tradicional	Infraestructura	Objetivo explícito del framework: FTP + MySQL/MariaDB + PHP 8+, sin Docker ni SSH
InfinityFree (y similares)	Proveedor de hosting	Mencionado explícitamente en .htaccess como caso objetivo (limitaciones de carpetas fuera de htdocs/)
Apache + mod_rewrite	Servidor web	.htaccess enruta todo el tráfico hacia public/index.php
Servidor embebido de PHP	Desarrollo local	bin/server.php + comando "misi serve"

3.5 Herramientas de desarrollo

Tecnología	Categoría	Propósito
Composer	Gestor de dependencias / autoload	Autoload PSR-4 (Misi\Business\, Misi\, App\, Modules\); el framework declara cero dependencias de terceros
CLI propia (bin/biz / misi)	Herramienta de línea de comandos	new, doctor, serve, db migrate/rollback/status/seed/fresh, make:*, create:business, info, route:list, config:list

install.sh / misi.cmd / misi.ps1	Instalador / wrapper multiplataforma	Instalación global del comando misi (Linux/macOS, Windows CMD y PowerShell)

3.6 Testing, CI/CD y servicios externos

No se encontró evidencia de un framework de pruebas (PHPUnit, Pest u otro), de archivos de configuración de integración continua (GitHub Actions, GitLab CI, etc.), ni de integraciones con servicios externos (pagos, mensajería, email, analítica). El proyecto no depende de ningún servicio de terceros para funcionar.

4. ARQUITECTURA DEL SISTEMA

4.1 Arquitectura general

Misi implementa una arquitectura en capas propia, descrita en su README como “Layered Architecture”, combinada con un patrón MVC simplificado (sin motor de plantillas: las vistas son archivos PHP incluidos con output buffering) y soporte de módulos opcionales. No es un framework full-stack de propósito general ni sigue un patrón formal como Clean Architecture o Hexagonal; es, en palabras del propio autor, un “Router HTTP minimalista” combinado con clases de soporte independientes (Database, Validator, Auth, Storage, Logger) orquestadas por una clase Application central.

4.2 Componentes principales

- Application (framework/Core/Application.php): punto de orquestación único. Carga variables de entorno y configuración, expone Router/Database/Validator/Session/Storage/Logger, registra middleware por defecto, descubre módulos, ejecuta el ciclo request/response y centraliza el manejo de errores.
- Router / Route (framework/Routing/): registra rutas GET/POST/PUT/PATCH/DELETE con parámetros {id} y un pipeline de middleware por alias.
- Database (framework/Database/Database.php): wrapper delgado sobre PDO — select/selectOne/insert/update/delete/transaction, con una barrera de validación de identificadores (assertSafeIdentifier) contra nombres de tabla/columna maliciosos.
- Auth (framework/Auth/Auth.php): autenticación basada en sesión y autorización por permisos vía tablas roles/permissions/role_user/permission_role.
- Validator (framework/Validation/Validator.php): validación de datos por reglas encadenables, incluyendo unique/exists contra la base de datos y reglas de archivos (file, image, mimes, max_size).
- Csrf (framework/Security/Csrf.php): token CSRF por sesión, validado por el middleware “csrf”.
- Storage / LocalStorage (framework/Storage/): abstracción de almacenamiento de archivos; único driver implementado es el sistema de archivos local.
- Logger (framework/Logging/Logger.php): log a archivo por día, con redacción automática de claves sensibles (password, token, secret, etc.).

- Business Core (business/): repositorios reutilizables (CustomerRepository, CategoryRepository, ProductRepository) con sus propias reglas de validación, pensados para copiarse entre proyectos.
- Módulos (modules/Catalog, modules/Example): funcionalidad aislada con sus propios controladores, rutas y, opcionalmente, migraciones.

4.3 Comunicación entre componentes

Los controladores acceden a los servicios del framework mediante la función helper global app(), que expone la instancia activa de Application (patrón de acceso estático documentado explícitamente en el propio código como alternativa mínima a un contenedor de inyección de dependencias, dado que el Router instancia controladores con new $class() sin DI). El Router no soporta middleware con parámetros: las verificaciones de permisos específicos (por ejemplo, categories.manage) se realizan de forma explícita dentro de cada controlador, no en el middleware.

4.4 Flujo general de datos

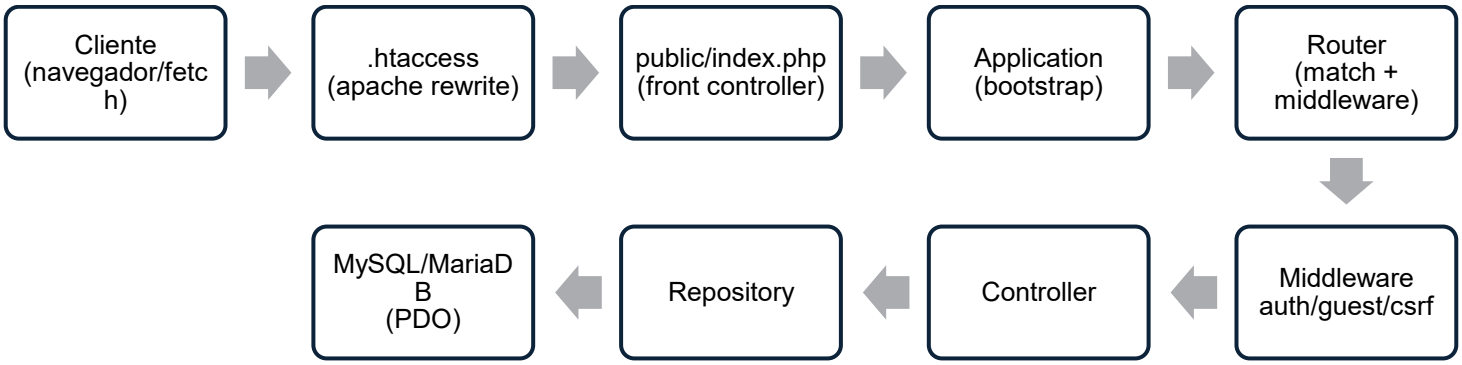


Figura 1. Ciclo de vida de una petición HTTP en Misi Framework.

Toda petición entra por .htaccess, que reescribe hacia public/index.php (front controller). Este carga bootstrap/app.php, que instancia Application y registra las rutas de routes/web.php y de cada módulo descubierto. Application::run() captura el Request, lo despacha a través del Router (que resuelve la ruta, aplica el pipeline de middleware y llama al controlador) y, si ocurre una excepción en cualquier punto, Application::handleException() la traduce a una respuesta JSON o HTML apropiada.

4.5 Arquitectura en capas del proyecto

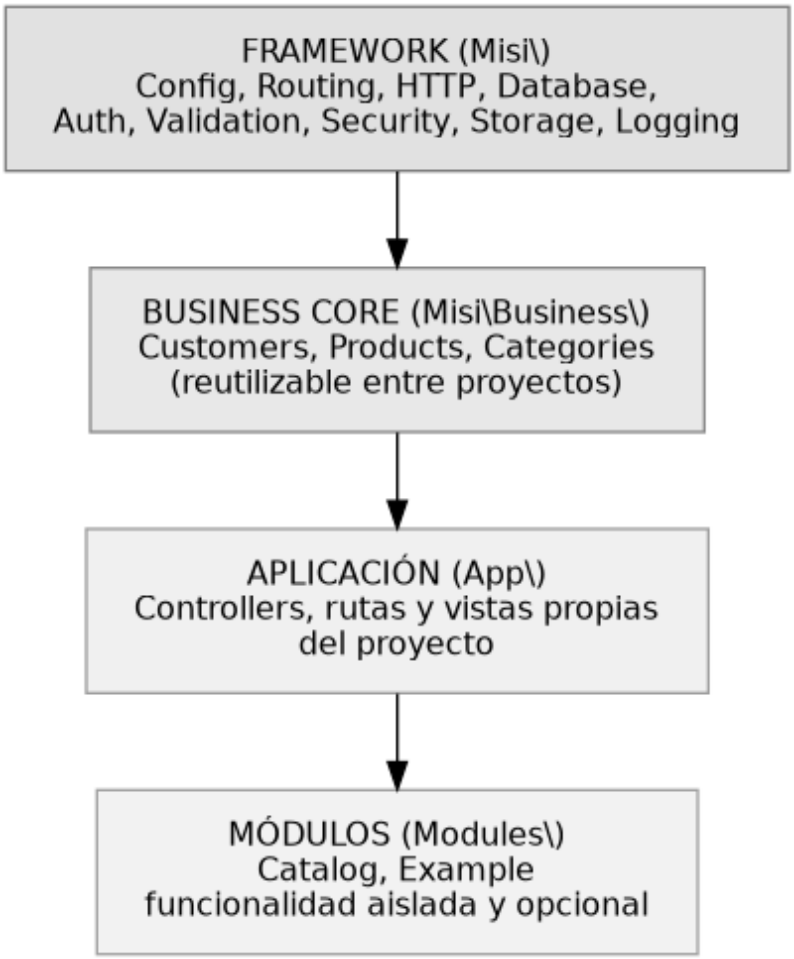


Figura 2. Capas declaradas por el propio README de Misi Framework.

4.6 Decisiones arquitectónicas identificables

- Sin ORM ni Query Builder: SQL crudo directo, documentado como decisión deliberada para mantener el código legible por una desarrolladora junior/intermedia y evitar abstracción sin necesidad real comprobada.
- Sin contenedor de inyección de dependencias: se prefiere el acceso estático vía app() mientras el número de dependencias por controlador se mantenga bajo.
- CLI en un único archivo (bin/biz), explícitamente no reestructurada como arquitectura de comandos orientada a objetos (Command + Kernel + Registry) por considerarse sobre-ingeniería para ~20 comandos.
- Migraciones aditivas: los cambios de esquema se agregan como nuevos archivos de migración en vez de editar migraciones ya aplicadas (ver migración 004, que añade un índice en lugar de modificar la migración 002 original).
- Cabeceras de seguridad por defecto aplicadas en un único punto (Application::applyDefaultSecurityHeaders), para que ningún controlador individual tenga que recordarlas.

5. ESTRUCTURA DEL PROYECTO

Árbol relevante del repositorio analizado (se omiten archivos `.gitkeep` y artefactos generados):

```
misi-framework/
├── framework/
│   ├── Core/Application.php
│   ├── Routing/
│   ├── Database/
│   ├── Http/
│   ├── Auth/Auth.php
│   ├── Security/Csrf.php
│   ├── Validation/Validator.php
│   ├── Storage/
│   ├── Logging/Logger.php
│   ├── Exceptions/
│   └── Support/
│
├── business/
│   ├── Customers/CustomerRepository.php
│   ├── Products/
│   └── migrations/
│
├── app/
│   └── Http/Controllers/
│
├── modules/
│   ├── Catalog/
│   └── Example/
│
├── config/
├── database/
│   ├── migrations/
│   └── seeders/DatabaseSeeder.php
├── routes/web.php
├── bootstrap/
├── public/
├── resources/
│   ├── views/
│   ├── scaffold/
│   └── stubs/
├── storage/
├── bin/
├── misi.cmd / misi.ps1
├── install.sh
├── .htaccess
├── composer.json
└── README.md
```

Elemento	Tipo	Propósito
`framework/`	Directorio	Infraestructura técnica reutilizable del framework (namespace Misi\)

`business/`	Directorio	Repositorios de dominio reutilizables entre proyectos (namespace Misi\Business\)
`app/`	Directorio	Código específico de la aplicación de referencia (namespace App\)
`modules/`	Directorio	Funcionalidad aislada y opcional (namespace Modules\)
`config/`	Directorio	Archivos de configuración, uno por dominio (app, database, logging, session, storage)
`database/migrations/`	Directorio	Migraciones del núcleo (usuarios, roles/permisos, uploads)
`routes/web.php`	Archivo	Registro de rutas raíz de la aplicación
`bootstrap/app.php`	Archivo	Construye la instancia de Application y carga las rutas
`public/index.php`	Archivo	Front controller — único punto de entrada HTTP
`bin/biz`	Archivo (CLI)	Implementación completa de la CLI de desarrollo (1213 líneas)
`.htaccess`	Archivo	Reglas de reescritura de Apache adaptadas a hosting compartido sin carpetas fuera de htdocs/

6. ARQUITECTURA DEL FRONTEND

El proyecto incluye frontend, pero limitado a HTML/CSS/JavaScript sin framework ni motor de plantillas. Las vistas (resources/views/*.php, modules/Catalog/Views/panel.php) son archivos PHP planos incluidos por el controlador mediante output buffering (ob_start() / require / ob_get_clean()) — no existe un sistema de plantillas tipo Blade o Twig.

Organización de páginas

- resources/views/welcome.php (628 líneas) — página de bienvenida de desarrollo, servida por WelcomeController.
- resources/views/ui-kit.php (352 líneas) — referencia viva de componentes visuales, servida por UiKitController.
- modules/Catalog/Views/panel.php — panel administrativo del módulo Catalog, servido por PanelController.
- public/views/*.html (misi.html, routes.html, ui-kit.html) — páginas estáticas de marketing del propio framework, generadas por bin/build-landing.php a partir de public/views/_partials/ y _content/ (no incluidos en el material analizado).

Routing de vistas

El enrutamiento de páginas HTML sigue las mismas reglas que cualquier otra ruta GET del Router (framework/Routing/Router.php). Las páginas de marketing estáticas se sirven directamente por Apache mediante reglas específicas en .htaccess, sin pasar por PHP.

Componentes, formularios y validación

No existe un sistema de componentes reutilizables del lado del cliente ni gestión de estado en JavaScript. La validación de formularios ocurre exclusivamente en el servidor, vía Validator; el JavaScript (public/js/api.js, ui.js, uikit.js, catalog.js) se limita a interacción de interfaz y llamadas fetch/AJAX contra la API.

Cliente API y assets

public/js/api.js concentra las llamadas HTTP hacia el backend (incluyendo el manejo del token CSRF vía el endpoint GET /api/csrf-token y el header X-CSRF-Token). Los estilos viven en un único archivo, public/css/misi.css (344 líneas), sin preprocesador (Sass/Less) ni sistema de utilidades tipo Tailwind.

7. ARQUITECTURA DEL BACKEND

El backend sigue una organización por capas simple: Controladores (delgados) → Repositorios de Business Core (cuando aplica) → Database (PDO). No hay una capa de Servicios genérica ni Repositorios para todos los recursos — solo Business Core (Customers, Categories, Products) tiene repositorios; los controladores de demostración en app/Http/Controllers/Api/ acceden a Database y Storage directamente.

Controladores

Los controladores son clases simples sin interfaz común obligatoria, instanciadas por el Router con new $class(). Se identificaron dos patrones: controladores que sirven una vista PHP con output buffering (WelcomeController, UiKitController, PanelController) y controladores de API que devuelven JsonResponse (PingController, ValidationDemoController, AuthDemoController, UploadDemoController, y los del módulo Catalog).

Middleware

Application::registerDefaultMiddleware() registra tres alias disponibles desde el primer momento en cualquier proyecto: auth (exige Auth::check(), lanza AuthenticationException si falla), guest (exige que NO haya sesión activa, lanza AuthorizationException si la hay) y csrf (valida el token CSRF en métodos que mutan estado; los métodos de solo lectura —GET, HEAD, OPTIONS— quedan exentos). Cada proyecto puede sobrescribir estos alias registrando de nuevo aliasMiddleware() con el mismo nombre.

Manejo de errores

Application::handleException() centraliza el manejo de excepciones: ValidationException se traduce a un 422 con errores estructurados por campo sin registrarse en el log (se considera tráfico normal); las HttpException con código ≥ 500 se registran como error; cualquier excepción no controlada se registra como critical y, en modo debug, se muestra con detalle; en producción se devuelve un genérico “Internal Server Error” (500).

Procesos en segundo plano

No se encontró evidencia de colas, workers persistentes ni tareas programadas (cron) en el código analizado — coherente con el objetivo declarado de funcionar en hosting compartido sin procesos persistentes.

8. MÓDULOS DEL SISTEMA

Application::discoverModules() escanea modules/{Nombre}/module.php (o backend/modules/ en un proyecto generado con “misi new”) y registra el nombre, archivo de rutas y carpeta de migraciones de cada módulo encontrado. Se identificaron dos módulos en el repositorio.

8.1 Módulo Catalog

Propósito

Primer módulo de negocio real del framework: un panel administrativo de catálogo (categorías y productos) sobre las tablas de Business Core, con control de acceso por permisos.

Componentes principales

- PanelController — sirve la vista del panel (modules/Catalog/Views/panel.php).
- CategoryController — CRUD de categorías.
- ProductController — CRUD de productos y ajuste de inventario (adjustStock).

Servicios y datos involucrados

Consume CategoryRepository y ProductRepository de Business Core (business/Products/). No declara migraciones propias: las tablas categories y products pertenecen a Business Core.

Endpoints relacionados

Método	Ruta	Middleware	Permiso adicional
GET	`/modules/catalog/panel`	auth	—
GET	`/modules/catalog/categories`	auth	— (lectura libre para autenticados)
POST	`/modules/catalog/categories`	auth, csrf	categories.manage
PUT	`/modules/catalog/categories/{id}`	auth, csrf	categories.manage
DELETE	`/modules/catalog/categories/{id}`	auth, csrf	categories.manage
GET	`/modules/catalog/products`	auth	—
GET	`/modules/catalog/products/{id}`	auth	—
POST	`/modules/catalog/products`	auth, csrf	products.manage
PUT	`/modules/catalog/products/{id}`	auth, csrf	products.manage
DELETE	`/modules/catalog/products/{id}`	auth, csrf	products.manage
POST	`/modules/catalog/products/{id}/stock`	auth, csrf	inventory.manage

8.2 Módulo Example

Propósito

Módulo de referencia sin lógica de negocio real: demuestra el contrato mínimo de module.php (name, routes, migrations) y que el Migrator recorre las migraciones de un módulo con el prefijo “Example/” automáticamente.

Componentes principales

- PingController — endpoint de verificación (GET /modules/example/ping).
- Carpetas vacías (.gitkeep) para Models/, Repositories/, Services/ y Views/ — presentes como plantilla, sin contenido real.
- migrations/001_create_example_pings_table.php — migración de ejemplo.

9. BASE DE DATOS

9.1 Tecnología

MySQL/MariaDB, acceso vía PDO con sentencias preparadas (framework/Database/Database.php). Charset utf8mb4, motor InnoDB en todas las tablas creadas por migraciones.

9.2 Modelo general

No hay ORM: cada migración define su tabla con SQL DDL crudo dentro de un método up()/down(). Las entidades se agrupan en dos orígenes: el núcleo del framework (autenticación y archivos) y Business Core (dominio de negocio reutilizable: clientes, categorías, productos).

9.3 Entidades principales y relaciones

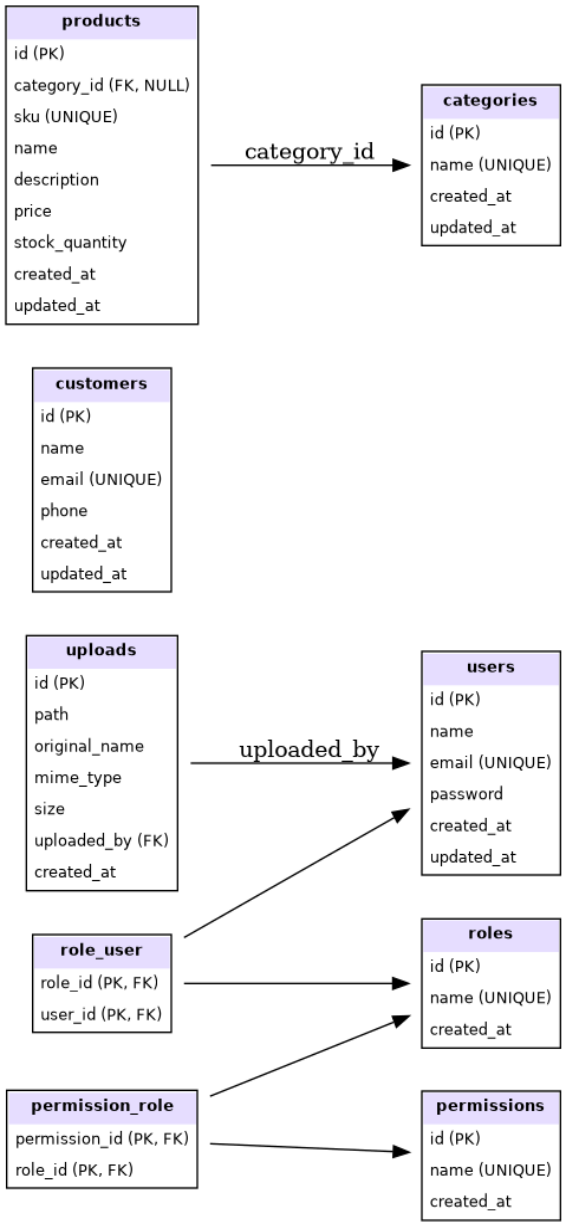


Figura 3. Modelo entidad-relación de las tablas identificadas en las migraciones.

9.4 Tablas o colecciones

Tabla	Origen (migración)	Propósito
Users	database/migrations/001	Usuarios del sistema para autenticación
roles / permissions / role_user / permission_role	database/migrations/002	RBAC simple: roles, permisos y sus tablas pivote

uploads	database/migrations/003	Metadata de archivos subidos (el archivo en sí vive en Storage, nunca como BLOB)
customers	business/migrations/001	Clientes (Business Core, reutilizable entre proyectos)
categories	business/migrations/002	Categorías de producto (Business Core)
products	business/migrations/003	Catálogo de productos con stock simple (Business Core)
migrations	Creada automáticamente por Migrator::ensureMigrationsTable()	Registro de migraciones ejecutadas (id, migration, batch, run_at)

9.5 Campos importantes, claves y restricciones

- users.email tiene restricción UNIQUE; users.password almacena un hash (password_hash(), nunca texto plano).
- role_user y permission_role usan claves primarias compuestas y ON DELETE CASCADE en sus foreign keys.
- uploads.uploaded_by referencia a users(id) con ON DELETE SET NULL.
- products.category_id referencia a categories(id) con ON DELETE SET NULL — borrar una categoría no bloquea ni elimina en cascada sus productos.
- products.sku y customers.email y categories.name tienen restricción UNIQUE.

9.6 Migraciones

El Migrator (*framework/Database/Migrator.php*) combina varias fuentes en orden: el núcleo (database/migrations/), Business Core (business/migrations/, si la carpeta existe) y cada módulo que declare carpeta de migraciones propia. El identificador guardado en la tabla migrations incluye el nombre de la fuente como prefijo para las no-core (por ejemplo, “Business/001_create_customers_table.php”), evitando colisiones entre migraciones numeradas igual en distintos orígenes. Se identificó una migración correctiva (004_add_role_user_user_id_index) que agrega un índice de rendimiento sin modificar la migración original, documentando explícitamente el principio de “migraciones aditivas” del proyecto.

9.7 Seeders

database/seeders/DatabaseSeeder.php crea un usuario administrador de demostración (admin@misi.test / changeme, con contraseña hasheada) y le asigna el rol “admin” con el permiso “users.manage”. Se ejecuta con “php bin/biz db:seed” o “misi db seed”.

9.8 Índices y optimizaciones

La migración 004 documenta un hallazgo de rendimiento real: la clave primaria compuesta de role_user (role_id, user_id) no cubre por prefijo izquierdo las consultas que filtran solo por user_id (usadas en Auth::can() en cada verificación de permiso), por lo que se agregó un índice dedicado idx_role_user_user_id. El método down() de esa migración es intencionalmente un no-op, documentado en el propio código, porque InnoDB reutiliza ese índice para la foreign key hacia users(id) y no puede eliminarse sin antes eliminar dicha foreign key.

10. API

10.1 Descripción general

La API es JSON sobre HTTP, sin versión de ruta explícita (no hay prefijo /v1/ ni similar) y sin una especificación OpenAPI/Swagger en el repositorio. Todas las rutas de API viven bajo el prefijo /api/ o /modules/{nombre}/ para las expuestas por módulos.

10.2 Convenciones

Toda respuesta JSON sigue un formato estándar definido por JsonResponse: en éxito, { "success": true, "data": ..., "message": "..." }; en error, { "success": false, "data": null, "message": "...", "errors": {} }.

10.3 Autenticación

Basada en sesión (cookie), no en tokens portables tipo JWT. Las rutas protegidas usan el middleware auth; las rutas exclusivas para visitantes no autenticados usan guest. Las rutas que mutan estado además requieren el middleware csrf (excepto los métodos de solo lectura).

10.4 Endpoints (rutas raíz — routes/web.php)

Método	Ruta	Middleware	Propósito
GET	/	—	Página de bienvenida (WelcomeController::index)
GET	/saludo/{name}	—	Ruta de ejemplo con parámetro (WelcomeController::greet)
GET	/ui-kit	—	Referencia visual de componentes (UiKitController::index)
GET	/api/ping	—	Verificación de que la API responde (PingController)
POST	/api/validate-demo	—	Demostración del Validator (ValidationDemoController)
POST	/api/login	guest	Inicio de sesión (AuthDemoController::login)
POST	/api/logout	auth, csrf	Cierre de sesión (AuthDemoController::logout)
GET	/api/me	auth	Usuario autenticado actual + permiso users.manage (AuthDemoController::me)
GET	/api/csrf-token	—	Entrega el token CSRF para clientes fetch/AJAX
POST	/api/uploads	auth, csrf	Subida de archivo (UploadDemoController::store)
DELETE	/api/uploads/{id}	auth, csrf	Elimina un archivo subido (UploadDemoController::destroy)
GET	/storage/{path*}	—	Sirve un archivo almacenado (UploadDemoController::show)

10.5 Manejo de errores

Los códigos de estado se derivan de la excepción lanzada: 401 (AuthenticationException, no autenticado), 403 (AuthorizationException, sin permiso), 404 (NotFoundException), 419 (token CSRF inválido o ausente — convención tomada explícitamente de Laravel), 422 (ValidationException o InsufficientStockException) y 500 (DatabaseException u otra excepción no controlada).

11. AUTENTICACIÓN Y AUTORIZACIÓN

11.1 Método

Autenticación basada en sesión PHP nativa (framework/Auth/Auth.php), sin JWT, sin OAuth y sin "recordarme" persistente — explícitamente fuera de alcance según los comentarios del propio código, salvo que un proyecto real lo requiera (por ejemplo, una app móvil sin cookies).

11.2 Flujo

- Auth::attempt(email, password) busca el usuario por email y verifica el hash con password_verify().
- Si es válido, Auth::login() regenera el ID de sesión (mitigación de session fixation) antes de guardar el ID de usuario en sesión.
- Auth::user() resuelve y cachea el usuario autenticado por request, excluyendo siempre el campo password de la respuesta.
- Auth::logout() destruye la sesión por completo (Session::destroy()).

11.3 Autorización (RBAC)

Auth::can(permiso) consulta, mediante un JOIN entre permissions, permission_role, role_user, si el usuario autenticado tiene asignado (a través de algún rol) el permiso solicitado. La convención de nombre es "recurso.accion" (por ejemplo, categories.manage, products.manage, inventory.manage, users.manage). No hay jerarquía de roles ni permisos condicionales.

11.4 Protección de rutas y middleware

Los middleware auth y guest protegen rutas completas. Las verificaciones de permisos específicos (más granulares que "autenticado o no") se hacen dentro del propio controlador, porque el Router no soporta middleware con parámetros — decisión documentada explícitamente en el código de los controladores de Catalog.

11.5 Cookies de sesión

Session::configure() fija httponly=true siempre, secure según configuración (forzado a true en producción por config/session.php) y SameSite configurable (Lax por defecto). El nombre de la cookie es configurable vía SESSION_COOKIE.

12. INTEGRACIONES EXTERNAS

No se encontró evidencia de integraciones con servicios externos (pasarelas de pago, mensajería, email transaccional, analítica, autenticación social) en el código fuente analizado. El framework está diseñado explícitamente para funcionar sin dependencias externas ni conexión a internet más allá de la propia base de datos.

13. TESTING

No se encontró ninguna carpeta de pruebas (tests/), ningún framework de testing declarado en composer.json (ni PHPUnit ni Pest) y ningún script de ejecución de pruebas. El comando “misi doctor” realiza diagnósticos de entorno (ver sección 16), pero no ejecuta pruebas automatizadas del código de la aplicación.

14. BUILD Y DESPLIEGUE

14.1 Proceso de build

No hay paso de build en el sentido de compilación o transpilación: el código PHP se ejecuta directamente y no hay assets de frontend que requieran bundling. bin/build-landing.php genera las páginas HTML estáticas de marketing (public/views/*.html) a partir de plantillas (“partials” y “content”) no incluidas en el material analizado, pero ese proceso pertenece al propio sitio del framework, no a un proyecto creado con él.

14.2 Despliegue objetivo: hosting compartido

El despliegue documentado es FTP + MySQL/MariaDB + PHP 8+, sin Docker, SSH ni procesos persistentes. El .htaccess incluido está adaptado específicamente para el plan gratuito de InfinityFree, donde la cuenta queda “enjaulada” dentro de htdocs/ y no se pueden crear carpetas hermanas fuera de ese directorio; por eso todo el proyecto (incluyendo config/, framework/, business/, storage/ y .env) vive dentro del propio htdocs/, y es el propio .htaccess el que impide que esos archivos se sirvan directamente por el navegador (bloqueo de dotfiles y de cualquier ruta que no resuelva a un archivo real dentro de public/).

14.3 Variable DEPLOY_TOKEN

.env.example incluye una variable DEPLOY_TOKEN, documentada como “solo necesaria en producción, y solo mientras el deploy-runner esté subido al hosting (hosting sin SSH)”, con instrucción de generarla mediante bin2hex(random_bytes(32)). No se encontró en el material analizado el script “deploy-runner” que consume esta variable, por lo que no fue posible determinar su implementación exacta a partir del código disponible.

14.4 Migraciones de producción

Se ejecutan con el mismo comando que en desarrollo (php bin/biz migrate o misi db migrate), sobre la conexión configurada en el .env del entorno de destino. No se encontró evidencia de un pipeline de CI/CD que las automatice.

15. CONFIGURACIÓN DEL PROYECTO

15.1 Requisitos previos

- PHP 8.1 o superior (composer.json exige >=8.1; el README indica “PHP 8+”).
- Extensión PHP mbstring habilitada (verificada explícitamente al arrancar; el proceso termina con un mensaje claro si falta).
- Extensión PDO / pdo_mysql disponible (requerida en tiempo de ejecución, aunque no está declarada como “require” en composer.json).
- Servidor MySQL o MariaDB accesible.
- Composer (opcional): si vendor/autoload.php no existe, bootstrap/autoload.php cae a un autoloader PSR-4 manual propio, así que el proyecto también funciona sin “composer install” en hosting sin acceso a internet.

15.2 Dependencias

El framework declara cero dependencias de terceros en composer.json ("require": solo "php" y "ext-mbstring"). Esto es una decisión de diseño explícita para minimizar puntos de fallo en hosting compartido.

15.3 Variables de entorno

Variable	Propósito	Obligatoria	Ejemplo (de .env.example)
APP_NAME	Nombre de la aplicación	No (default: Misi)	Misi
APP_ENV	Entorno de ejecución	No (default: development en .env.example)	development
APP_DEBUG	Muestra detalle de errores 500 si es true	No (default: true en .env.example)	true
APP_URL	URL base de la aplicación	No	http://localhost:8000
APP_TIMEZONE	Zona horaria de PHP	No (default: America/Bogota en .env.example)	America/Bogota
DB_CONNECTION	Motor de base de datos	No (default: mysql)	mysql
DB_HOST	Host de la base de datos	No (default: 127.0.0.1)	127.0.0.1
DB_PORT	Puerto de la base de datos	No (default: 3306)	3306
DB_DATABASE	Nombre de la base de datos	Sí, en la práctica	misi
DB_USERNAME	Usuario de la base de datos	Sí, en la práctica	root
DB_PASSWORD	Contraseña de la base de datos	Según entorno	(vacío en el ejemplo)
STORAGE_DRIVER	Driver de almacenamiento	No (default: local; único implementado)	local
STORAGE_PATH	Ruta raíz de almacenamiento	No (default: storage/uploads)	storage/uploads
SESSION_COOKIE	Nombre de la cookie de sesión	No (default: misi_session)	misi_session
SESSION_LIFETIME	Duración de la cookie (0 = hasta cerrar navegador)	No	0

SESSION_SAME_SITE	Atributo SameSite de la cookie	No (default: Lax)	Lax
LOG_LEVEL	Nivel mínimo registrado por el Logger	No (default: debug)	debug
DEPLOY_TOKEN	Token para el proceso de despliegue en hosting sin SSH	Solo en producción, mientras el deploy-runner esté presente	(generar con bin2hex(random_bytes(32)))

15.4 Configuración local

Los archivos en config/ (app.php, database.php, logging.php, session.php, storage.php) leen sus valores desde Env::get(), que a su vez carga el archivo .env de la raíz del proyecto mediante un parser propio (framework/Support/Env.php), sin depender de vlucas/phpdotenv ni ninguna librería externa.

16. Instalación y ejecución

Pasos verificados contra bin/biz, install.sh y bootstrap/app.php:

16.1 Instalación global de la CLI (opcional)

```
# Instala el comando "misi" globalmente (Linux/macOS)
curl -fsSL https://raw.githubusercontent.com/emilymontec/misi-framework/main/install.sh | sh
```

16.2 Crear un proyecto nuevo

```
misi new <nombre-del-proyecto>
```

16.3 Configurar variables de entorno

```
cp .env.example .env
# Editar .env con las credenciales reales de MySQL/MariaDB
```

16.4 Ejecutar migraciones

```
misi db migrate
```

16.5 Cargar datos iniciales (opcional)

```
misi db seed
```

16.6 Ejecutar el proyecto en desarrollo

```
misi serve
```

16.7 Diagnóstico del entorno

```
misi doctor
```

17. CLI DE DESARROLLO

bin/biz (1213 líneas) implementa toda la lógica de la CLI en un único archivo, sin dependencias externas — decisión documentada explícitamente en el propio código, considerando una arquitectura de comandos orientada a objetos como sobre-ingeniería para el número de comandos actual. bin/misi es un wrapper en Bash que localiza el bin/biz del proyecto más cercano (buscando hacia arriba desde el directorio actual) o cae a una instalación global; misi.cmd y misi.ps1 cumplen el mismo rol en Windows.

Comando	Forma alternativa	Propósito
misi new <nombre>	—	Crea un proyecto nuevo a partir de resources/scaffold/
misi doctor	—	Diagnóstico del entorno actual (cada chequeo captura sus propios errores)
misi serve	*misi run*	Levanta el servidor embebido de PHP para desarrollo local
misi db migrate	*misi migrate*	Ejecuta las migraciones pendientes (núcleo + Business Core + módulos)
misi db rollback	*misi migrate:rollback*	Revierte el último lote (batch) de migraciones
misi db status	*misi migrate:status*	Muestra el estado (Ran/Pending) de cada migración
misi db seed	*misi db:seed*	Ejecuta database/seeders/DatabaseSeeder.php
misi db fresh	*misi db:fresh*	Revierte y vuelve a ejecutar todas las migraciones
misi make <tipo> <Nombre>	*misi make:<tipo> <Nombre>*	Generadores de código (controller, model, service, repository, module) desde resources/stubs/
misi create business <nombre>	*misi create:business*	Copia la carpeta business/ (Business Core) al proyecto
misi info	—	Información del proyecto
misi route:list	—	Lista las rutas registradas
misi config:list	—	Lista la configuración cargada
misi self-update	—	Actualiza la instalación global (git pull en $MISI_HOME)
misi help / misi	—	Referencia de comandos

18. INFRAESTRUCTURA Y OPERACIÓN

18.1 Almacenamiento de archivos

LocalStorage (framework/Storage/LocalStorage.php) guarda archivos bajo storage/uploads/ por defecto, con protecciones aplicadas siempre: nombre de archivo generado aleatoriamente (nunca el original), extensión saneada a solo caracteres alfanuméricos, bloqueo explícito de path traversal (rechaza cualquier ruta con "..") y permisos de archivo 0644 (nunca ejecutable). Solo hay un driver implementado; la interfaz StorageInterface deja preparada la extensión a S3/CloudStorage, explícitamente marcada como no implementada.

18.2 Logging

Logger (framework/Logging/Logger.php) escribe un archivo por día (storage/logs/misi-YYYY-MM-DD.log), con niveles debug/info/warning/error/critical y redacción automática de claves de contexto sensibles (password, token, secret, api_key, authorization, csrf, credit_card, cvv, entre otras) — reemplazadas por [REDACTED]. El propio código documenta que el stack trace completo se omite deliberadamente porque los argumentos de cada frame pueden contener valores sensibles que no pueden sanearse igual que las claves de contexto.

18.3 Monitoreo

No se encontró integración con ninguna herramienta de monitoreo o alerta externa (Sentry, New Relic, etc.). El logging a archivo plano es el único mecanismo de observabilidad identificado.

18.4 Networking / dominios

No se encontró configuración de dominio específica más allá de APP_URL en .env. El .htaccess fuerza HTTPS mediante una redirección 301 (RewriteCond %{HTTPS} off).

19. SEGURIDAD

19.1 Medidas implementadas

- SQL: todas las consultas usan sentencias preparadas de PDO; Database::assertSafeIdentifier() valida con regex que los nombres de tabla/columna interpolados en insert()/update()/delete() no contengan SQL incrustado (los identificadores no pueden pasarse como binding preparado en PDO).
- CSRF: token por sesión (no por formulario), comparado con hash_equals() para evitar timing attacks; aceptado tanto en el campo _token del body como en el header X-CSRF-Token.
- Sesión: cookies httponly siempre, secure forzado en producción, SameSite configurable; el ID de sesión se regenera en cada login/logout (mitigación de session fixation).
- Contraseñas: hasheadas con password_hash()/password_verify() (bcrypt/Argon2 según configuración de PHP); nunca se registran en el log ni se devuelven en Auth::user().
- Cabeceras HTTP de seguridad por defecto: X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN, Referrer-Policy: strict-origin-when-cross-origin, y Strict-Transport-Security cuando la conexión es HTTPS — aplicadas centralmente a toda respuesta.
- Subida de archivos: validación de tipo MIME real (finfo, no solo extensión), tamaño máximo, verificación de is_uploaded_file() antes de mover cualquier archivo, nombre de archivo siempre regenerado.
- Logging: redacción automática de claves sensibles en el contexto de cada entrada de log.

19.2 Ausencias explícitas (documentadas por el propio código)

- Sin Content-Security-Policy por defecto: decisión deliberada, documentada en Application.php, porque un CSP genérico razonable rompería proyectos con <script> inline sin nonce — se deja a criterio de cada proyecto.
- Un único token CSRF por sesión (no por formulario/petición), lo que no mitiga ataques tipo BREACH; considerado suficiente para el tamaño de proyecto objetivo.
- Sin rate limiting en ningún endpoint (incluido /api/login).
- Sin protección adicional contra fuerza bruta en el login más allá de lo que ofrezca el propio hosting.

19.3 Hallazgos técnicos y recomendaciones

ID	Hallazgo	Impacto	Prioridad	Recomendación
SEC-1	No hay límite de intentos (rate limiting) en POST	Facilita ataques de fuerza bruta contra	Alto	Agregar un límite de intentos por IP/usuario en el middleware de login

	/api/login ni en ningún otro endpoint.	credenciales de usuarios.		antes de exponer el proyecto a internet abierta.
SEC-2	APP_DEBUG=true y APP_ENV=development son los valores por defecto en .env.example.	Si se copia el .env.example a producción sin cambiarlo, los errores 500 se mostrarían con traza completa al visitante.	Medio	Verificar explícitamente APP_DEBUG=false y APP_ENV=production en cada despliegue; considerar que el instalador o "misi doctor" lo valide.
SEC-3	No hay Content-Security-Policy por defecto (decisión documentada).	Sin CSP, mitigaciones adicionales contra XSS quedan enteramente en manos de cada proyecto.	Bajo	Documentar en la guía de despliegue cómo cada proyecto debe agregar su propio CSP según los scripts/estilos que use.

20. DEUDA TÉCNICA Y HALLAZGOS

ID	Hallazgo	Impacto	Prioridad	Recomendación
DT-1	No existe ninguna suite de pruebas automatizadas (unitarias, integración o E2E) ni framework de testing declarado.	Cambios futuros en el framework o en Business Core no tienen red de seguridad automatizada; el riesgo de regresión recae enteramente en pruebas manuales.	Alto	Incorporar PHPUnit (o Pest) al menos para Database, Validator, Router y Auth, que son la base de todo lo demás.
DT-2	bootstrap/autoload.php mapea el namespace Misi\ (fuera de Misi\Business\) al directorio .misi/, pero en el repositorio analizado el código del framework vive en framework/ (así lo declara también composer.json: "Misi\\": "framework/").	Si vendor/autoload.php no existe (proyecto sin "composer install", el caso explícitamente soportado en hosting compartido), el autoloader manual de respaldo no encontraría las clases Misi* y el proyecto fallaría.	Alto	Verificar si esta es una discrepancia real entre la copia analizada y el repositorio activo, o si .misi/ y framework/ deben coexistir; alinear ambos mapeos.
DT-3	No se encontró ningún pipeline de CI/CD (GitHub Actions u otro) que ejecute linting, análisis estático o pruebas antes de un despliegue.	No hay verificación automatizada de calidad antes de que el código llegue a producción.	Medio	Agregar un workflow mínimo de CI que al menos ejecute "php -l" sobre los archivos modificados y, cuando exista, la suite de pruebas.
DT-4	El módulo Catalog depende de que el proyecto siembre manualmente los	Un proyecto que instale el módulo Catalog sin ajustar su propio seeder tendrá un panel	Medio	Documentar este requisito de forma explícita en el README del módulo, o proveer un seeder de

	permisos categories.manage, products.manage e inventory.manage; DatabaseSeeder.php solo siembra users.manage.	funcional pero con todas las acciones de escritura bloqueadas por falta de permisos asignados.		ejemplo específico para Catalog.
DT-5	El script bin/build-landing.php referencia plantillas en public/views/_partials/ y _content/ que no están presentes en el material analizado.	No fue posible verificar el funcionamiento completo del proceso de generación de las páginas de marketing.	Bajo	Confirmar si esas carpetas existen en el repositorio completo o si el script depende de contenido externo al framework en sí.

21. LIMITACIONES Y ÁREAS PENDIENTES

- El propio README enumera explícitamente features “congeladas” hasta que un proyecto real las necesite: Query Builder/ORM, más abstracciones de ruteo, más servicios de framework, tipos de módulo más especializados, infraestructura avanzada e integraciones no requeridas por la arquitectura actual.
- No se encontró documentación en un directorio docs/ (el README hace referencia a varios archivos como docs/architecture.md, docs/security.md, docs/routing.md, etc., pero ninguno estaba presente en el material analizado).
- No hay ningún archivo de ejemplo end-to-end (“examples/demo-app”) en el material recibido, aunque el código lo referencia repetidamente en comentarios como fuente de los patrones usados en Business Core.
- No se pudo determinar la versión exacta de PHP recomendada más allá del mínimo (8.1) declarado en composer.json.
- No fue posible determinar, únicamente a partir del código analizado, el contenido del script “deploy-runner” que consume la variable DEPLOY_TOKEN.

22. GLOSARIO

Término	Definición
Business Core	Capa de repositorios de dominio (Misi\Business\) pensada para copiarse entre proyectos que comparten el mismo tipo de entidades de negocio (clientes, categorías, productos).
Middleware alias	Nombre corto (auth, guest, csrf) registrado en el Router y asociado a una función que se ejecuta antes del controlador.
Migración aditiva	Cambio de esquema introducido como un nuevo archivo de migración, sin editar migraciones ya aplicadas en producción.
Módulo	Unidad de funcionalidad opcional y aislada (namespace Modules\) con sus propias rutas y, opcionalmente, migraciones, descubierta automáticamente por Application.

RBAC	Control de acceso basado en roles (Role-Based Access Control); en Misi se implementa con las tablas roles, permissions, role_user y permission_role.
Scaffold	Plantilla de proyecto (resources/scaffold/) que “misi new” copia para crear un proyecto nuevo.
Stub	Plantilla de código fuente (resources/stubs/) usada por los generadores “misi make:*”.

23. REFERENCIAS

- README.md del propio repositorio (fuente principal de la descripción funcional y filosofía del proyecto).
- composer.json — dependencias, autoload PSR-4 y versión del proyecto (1.4.0).
- .env.example — variables de entorno soportadas y sus valores por defecto.
- Código fuente de framework/, business/, app/, modules/, config/, database/, routes/, bootstrap/, public/ y bin/, analizado directamente para este documento.

RESUMEN FINAL DE LA AUDITORÍA

Campo	Detalle
Documento generado	Documentacion_Tecnica_MisiFramework.docx
Ubicación	/docs/generated/ (ver nota de ubicación equivalente más abajo)
Secciones generadas	24 secciones numeradas + portada, control del documento y resumen final
Módulos identificados	Catalog (panel administrativo de catálogo) y Example (módulo de referencia)
Tecnologías principales	PHP 8.1+, MySQL/MariaDB (PDO), HTML/CSS/JavaScript vanilla, sin dependencias de Composer de terceros
Diagramas generados	3 — arquitectura en capas, flujo de una petición HTTP, y modelo entidad-relación
Secciones omitidas por no aplicar	Ninguna integración externa (no existen); sin manual de usuario (excluido a propósito); sin CI/CD (no existe, documentado como hallazgo en vez de sección propia)
Principales limitaciones encontradas	Sin pruebas automatizadas; discrepancia entre el mapeo de autoload manual (.misi/) y la ubicación real del código (framework/); sin rate limiting en login; permisos de Catalog no sembrados automáticamente